Week 4: The Resilient System

IoT Bhai: RoboStart | Master Embedded C++

Mentors: Md. Kamruzzaman Tipu & Md. Shohanuzzaman

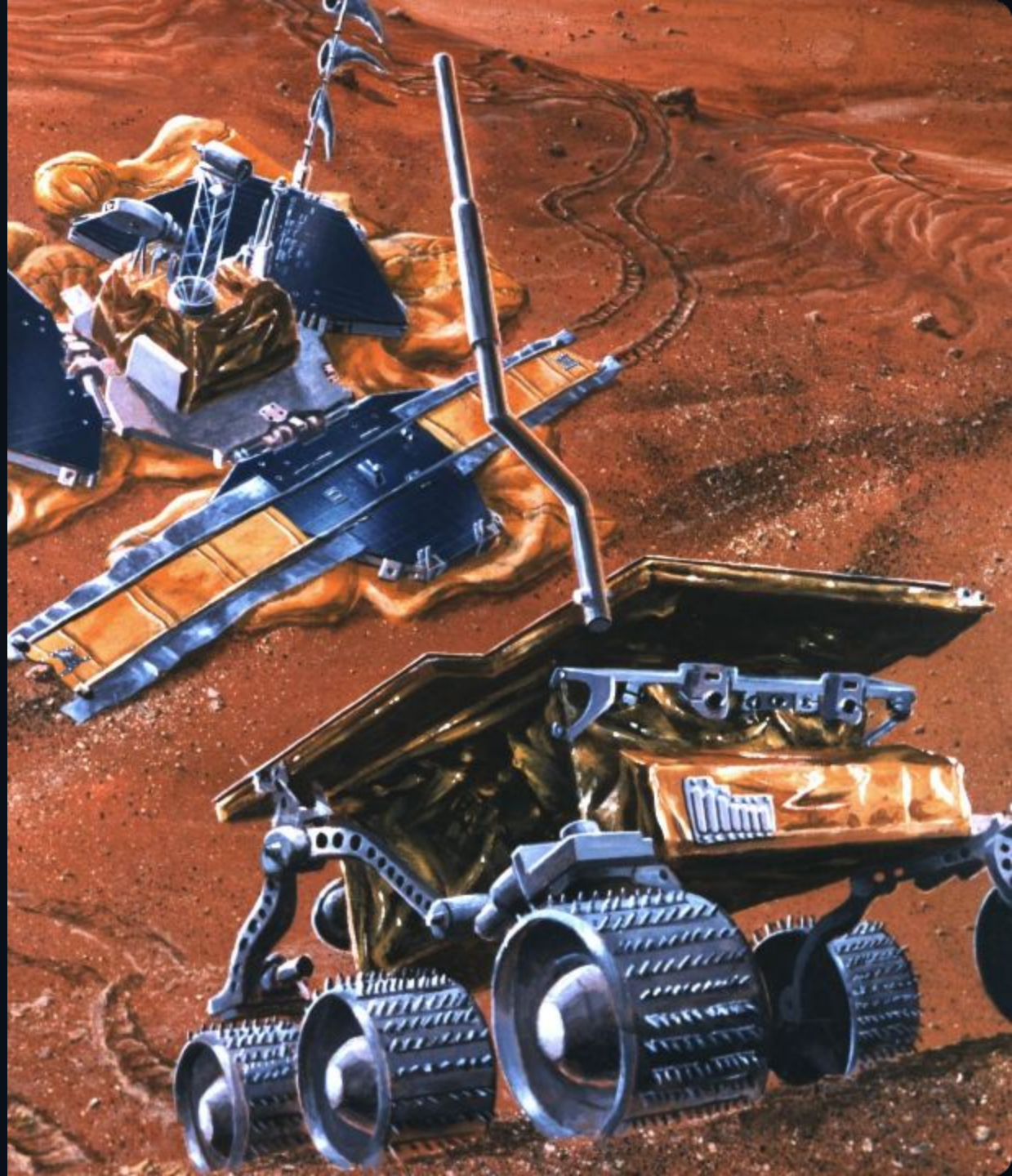
The Mars Pathfinder Crash

Priority Inversion (1997)

Shortly after landing on Mars, the $150 million Pathfinder rover began constantly rebooting itself, paralyzing the mission.

The problem? **Blocking code.** A low-priority meteorological task took too long to finish, which blocked a high-priority communication task. The system's internal watchdog assumed the computer was frozen and hit the reset button.

Today, we learn how to write aerospace-grade code. We will learn to write systems that *never* block, react instantly to danger, and heal themselves.



The delay() Function is Banned

Today, we stop writing "hobby" code and start writing "industrial" code.

A resilient system never sleeps, never ignores emergencies, and heals itself when it crashes.

Escaping the Time Trap

The delay() Coma

When you use **delay(1000)**, the Arduino is completely paralyzed for 1 entire second. It cannot read sensors, detect button presses, or avoid walls. It becomes effectively blind and deaf to the world around it.

The millis() Stopwatch

millis() acts like checking your watch. The Arduino keeps running all its tasks smoothly in the loop, simply glancing at its internal clock to see if it's time to trigger an event. This unlocks true multitasking!

What is ? `millis()`

- `millis()` total milliseconds passed since the Arduino board booted up.
- Think of it as an unstoppable internal stopwatch that begins ticking the exact moment power is applied.
- Unlike the traditional delay method, which pauses the processor, it allows your code to run continuously in a non-blocking loop.
- It is the foundational function required for true multitasking, reading sensors, and controlling hardware simultaneously.

The Blocking Problem

The Trap `delay()`

Using `delay(1000)` s the processor for one full second. The Arduino essentially goes to sleep and cannot read sensors, detect button presses, or run any other background tasks until the time has fully elapsed. It creates sequential, rigid programs.

The Advantage `millis()`

By tracking time, your logic becomes strictly non-blocking. Instead of pausing the processor, your loop runs freely. You simply check your "watch" to see if enough time has passed to trigger an action, allowing seamless and responsive multitasking.

The Three Pillars of Time

Current Time

Captured at the start of your loop using the function to see what the hardware stopwatch says right exactly now.

Previous Time

The exact timestamp of when you last performed the action. This state variable is updated every time the specific action fires.

Wait Interval

The defined duration you want to wait between actions, typically stored safely as a constant variable in milliseconds.

The Golden Rule

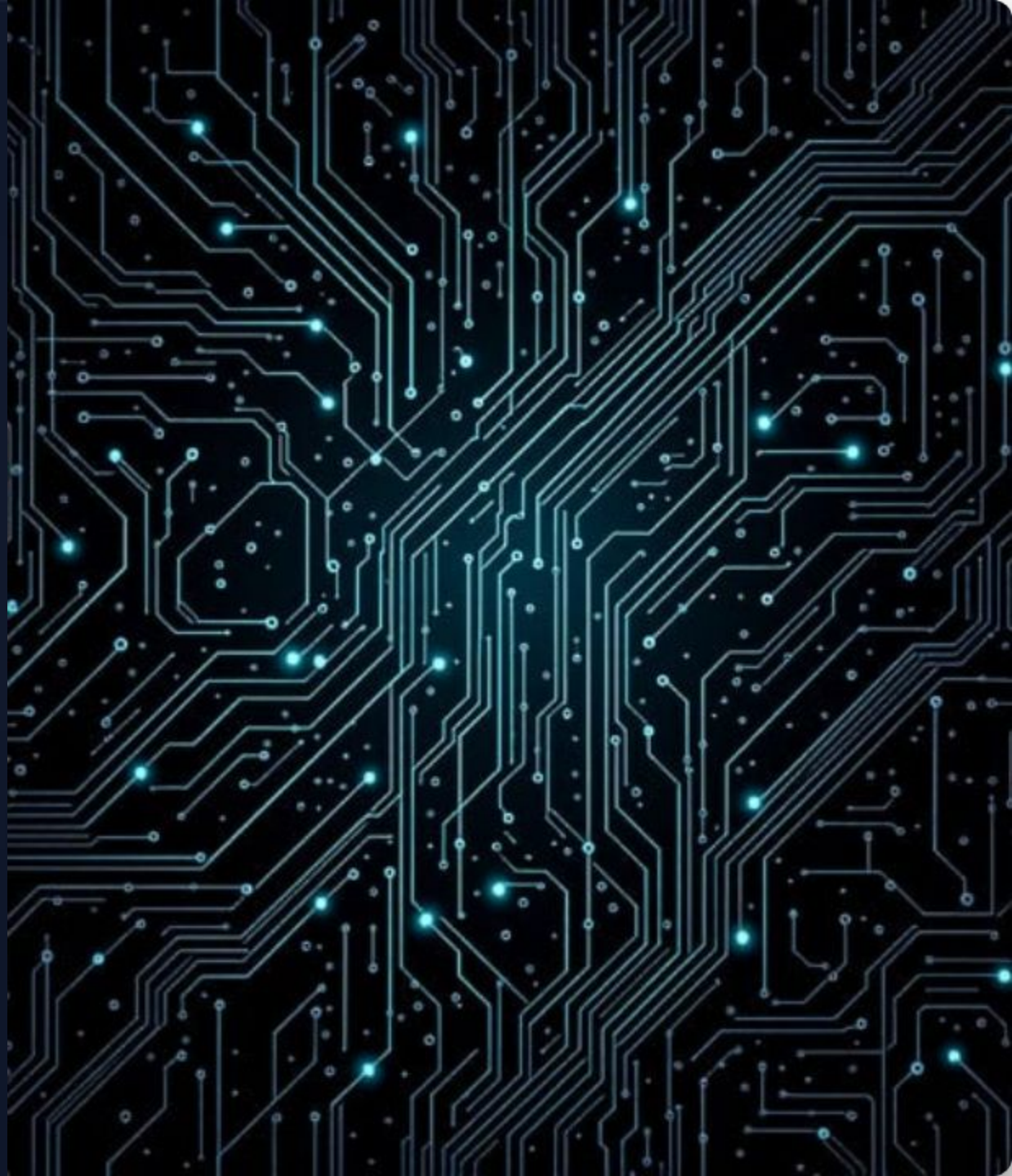
```
currentMillis - previousMillis ≥ interval
```

This strict subtraction logic is universally standard. It cleanly checks if the interval has passed without pausing the program, ensuring absolute timing accuracy.

Crucial Data Types

Because the hardware counts every millisecond, the numbers grow exponentially. You must always store these values in an variable type. `unsigned long`

If you accidentally use a standard integer, the variable will rapidly overflow and crash your program in just 32 seconds. An unsigned long provides vast capacity.



The 50-Day Rollover

- An `unsigned long` is a 32-bit integer, giving it a maximum capacity of exactly 4,294,967,295 milliseconds.
- Once the processor hits this exact number (roughly 49.7 days), the internal hardware counter overflows and resets to zero.
- By strictly using the subtraction method shown earlier, your code effortlessly survives this rollover without crashing.
- Binary unsigned integer math inherently guarantees the calculated time difference remains mathematically correct even after a complete reset!

Lab 1: True Multitasking

- **Wire the Hardware:** Connect 2 LEDs and 1 Push-Button to your Arduino breadboard.
- **The Heartbeat:** Write code to blink LED 1 exactly every 500ms using the millis() timing logic.
- **The Observer:** In the exact same loop(), read the button state digitally.
- **The Reaction:** If the button is pressed, turn LED 2 ON instantly. If released, turn it OFF.
- **The Aha Moment:** Watch as the button responds instantly to your touch while the heartbeat LED continues blinking flawlessly in the background!

Part 3: The Instant Override

Mastering Hardware Interrupts in Real-Time Systems

The 30-Millisecond Problem

In a standard loop, the CPU acts as a "Poller," checking tasks sequentially:

- Update Navigation Map
- Process High-Fidelity Audio
- Read External Sensors
- **Check Crash Sensor**

While checking music, the computer is blind to a crash.

The Crash: Danger of the Loop

The Limitation of "Polling"

Waiting your turn in the main `loop()` means processing tasks sequentially:

- GPS check... (10ms)
- Music check... (10ms)
- Tire check... (10ms)
- **CRASH HAPPENS HERE.**

The Fatal Result

If the deer jumps in front of the car right after the sensor was checked, a full 30 milliseconds pass before the computer checks it again.

In a car crash, 30ms is the difference between an airbag deploying in time and a catastrophic failure.

Interrupts: The Ultimate Solution

Hardware Priority

A hardware interrupt is a **physical signal** that commands the CPU to stop everything mid-sentence.

The crash sensor doesn't wait for its turn in the loop. It screams "STOP!" directly into the silicon.

> **Result:** Zero perceived lag. The airbag deploys at the microsecond of impact.

How Hardware Handles the Emergency

1. Freeze

The CPU remembers its exact spot in the loop(), like dropping a bookmark.

2. Jump

It instantly moves to the defined Interrupt Service Routine (ISR).

3. Execute

Handles the emergency code (e.g., deploying the airbag) as fast as possible.

4. Resume

Jumps back to the exact spot it left off as if nothing happened.

The Three-S Rules of the ISR

Short

Do one or two simple things and leave. The entire microcontroller is frozen while the ISR is running.

Simple

No delay() allowed. Delays rely on other interrupts, which are disabled, causing a permanent system lock-up.

Silent

No Serial.print(). Serial communication is slow and requires buffering. The CPU doesn't have time to chat.

Interrupt Trigger Modes

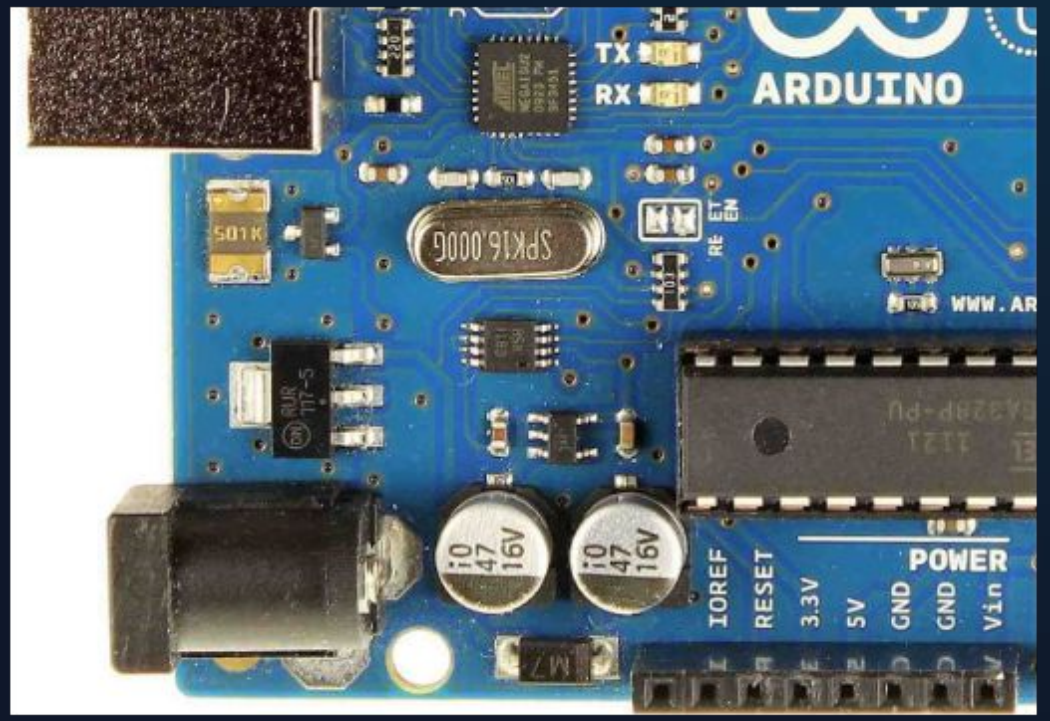
Mode	Behavior	Common Use Case
`LOW`	Triggers continuously as long as the pin is LOW.	Emergency kill switches or fail-safe sensors.
`CHANGE`	Triggers whenever the pin value toggles (HIGH to LOW *or* LOW to HIGH).	Reading rotary encoders or toggle switches.
`RISING`	Triggers only when the pin goes from LOW to HIGH.	Button presses (when using external pull-down resistors).
`FALLING`	Triggers only when the pin goes from HIGH to LOW.	Button presses (when using internal pull-up resistors).

Lab 2: The Emergency Stop

Building an instant hardware override circuit to bypass a lagging software loop.

Step 1: The Wiring

- Wire a standard push button directly to **Pin 2** and Ground.
- *Note:* Pin 2 is mapped to Hardware Interrupt 0 on the Arduino Uno.
- We will use the internal i`INPUT_PULLUP`o no external resistor is needed for the button.
- Wire a Red LED to **Pin 13** to serve as our emergency indicator light.



Step 2 & 3: Loop & Interrupt

The Terrible Loop

We will intentionally write a heavy, blocking . `loop()`

- Create a loop `for` that counts to 100.
- Print the count to the Serial Monitor.
- Add a massive `delay(500)` to simulate intense, slow processing.

The Interrupt Attach

In `setup()` the Arduino to listen for the emergency:

- Use: `attachInterrupt(digitalPinToInterrupt(2), emergencyOverride, FALLING);`
- This triggers our custom function the instant the pin voltage falls to LOW.

`emergencyOverride`

Step 4: The ISR Function

Delays Allowed

Writing the Override

Create the function: `void emergencyOverride()`

Inside this function, you will do exactly two things:

1. Toggle a variable. `volatile bool emergencyState`
2. Use to flip the LED. `digitalWrite(13, emergencyState)`

That is it. Fast, efficient, and reliable.

The Real-World Lesson

You will see the red LED toggle instantly, even while the Serial Monitor is lagging and counting slowly.

THE TRUE POWER OF AN INTERRUPT

Ready to Code?

"A resilient system is built for the worst-case scenario."

IoT Bhai: RoboStart | Dhaka, Bangladesh